PHB 243b Lecture 4: Speaker Notes

Version Control with Git and GitHub

Dr. Ronald G. Thomas

2026-01-15

Table of contents

1 Course Information and Welcome **2**

2 Today's Agenda Overview **2**

3 The File Naming Problem **3**

4 What is Version Control? **3**

5 Git: The Version Control Standard **4**

6 Core Git Concepts **4**
- 6.1 Repository (repo) . . . 4
- 6.2 Commit . . . 4
- 6.3 Branch . . . 5
- 6.4 Merge . . . 5
- 6.5 Remote . . . 5

7 Basic Git Workflow **5**
- 7.1 Checking Status . . . 5
- 7.2 Staging Changes . . . 5
- 7.3 Committing . . . 6

8 Writing Good Commit Messages **6**
- 8.1 Format . . . 6
- 8.2 Content . . . 6

9 Introduction to GitHub **6**
- 9.1 What GitHub Adds . . . 7
- 9.2 Alternatives . . . 7

10 Connecting to GitHub **7**
- 10.1 Cloning . . . 7
- 10.2 Adding a Remote . . . 7
- 10.3 Push and Pull . . . 8

11 Collaboration Workflow **8**
- 11.1 Why Branches? . . . 8
- 11.2 Pull Requests . . . 8

12 What to Track with Git **8**
12.1 DO Track 9
12.2 DO NOT Track 9
12.3 The .gitignore File 9
12.4 Handling Large Data 9

13 Summary **9**
13.1 For Your Course Projects 10
13.2 Coming Up 10

14 Additional References **10**
14.1 Primary Documentation 10
14.2 Books and Tutorials 10
14.3 Academic Papers 10
14.4 Tools and Extensions 11

1 Course Information and Welcome

Welcome to lecture 4 of PHB 243b. Today we cover version control with Git and GitHub, essential skills for any modern data scientist or biostatistician. Version control tracks every change to your files, enables collaboration without conflicts, and provides a complete audit trail of your analytical decisions. These skills are expected in both academic research and industry positions.

By the end of today's lecture, you will understand Git fundamentals and be ready to use GitHub for your course projects. The skills learned today connect directly to Tuesday's lecture on renv and Docker—together, these tools form the foundation of reproducible research infrastructure.

Version control is not merely a convenience; it is increasingly a professional requirement. Journals such as *Nature* and *PLOS* expect code availability, and funding agencies including NIH emphasize reproducibility in grant proposals. GitHub profiles have become de facto portfolios for data scientists, and familiarity with Git is assumed in most job postings involving data analysis.

Technical note: We focus on command-line Git because understanding the underlying commands makes troubleshooting easier. However, RStudio provides a graphical Git interface that many find more approachable for daily use. Both approaches use the same underlying system—skills transfer directly.

2 Today's Agenda Overview

We proceed through six interconnected topics:

1. **Why Version Control?**: The problems with manual file versioning and how version control solves them systematically.
2. **Git Fundamentals**: Core concepts including repositories, commits, branches, and merges—the vocabulary you need to understand Git.
3. **Basic Git Workflow**: The edit-stage-commit cycle that forms the foundation of daily Git use.
4. **Introduction to GitHub**: Cloud hosting for Git repositories and the collaboration features it provides.
5. **Collaboration with GitHub**: Professional workflows using branches and pull requests for team-based projects.
6. **Best Practices for Biostatistics Projects**: What to track, what to exclude, and how to handle sensitive data.

The opening question about how students currently track file versions invariably produces recognition and sometimes embarrassment. The "analysis_final_FINAL_v2.R" pattern is universal—nearly every researcher has experienced this chaos. This shared experience motivates the solution.

Reference: Ram K. (2013). Git can facilitate greater reproducibility and increased transparency in science. *Source Code for Biology and Medicine* 8:7. This early paper articulating Git's value for scientific research remains relevant.

3 The File Naming Problem

The slide showing chaotic file names resonates because it depicts reality. A 2018 survey of researchers found that 89% had experienced confusion about which version of a file was current, and 67% had accidentally overwritten collaborators' work.

The problems with manual versioning include:

- **Ambiguity**: File names cannot capture what changed or why. "analysis_v2" tells you nothing about the nature of the changes.
- **Proliferation**: Projects accumulate dozens of files, most of which will never be opened again but cannot be safely deleted.
- **Collaboration failure**: When two people work simultaneously, someone's changes get lost. Email-based collaboration ("here's my version") makes this worse.
- **No audit trail**: You cannot reconstruct the sequence of decisions that led to the current state. Regulatory contexts may require this history.

The student question about combining simultaneous edits highlights the core collaboration problem. Without tooling, merging changes requires tedious manual comparison. Even careful researchers make mistakes, and lost work damages both productivity and morale.

Technical note: Some researchers attempt systematic naming conventions like "analysis_20260115_rgt.R" (date and initials). While better than arbitrary names, this still lacks the ability to see what changed or merge concurrent work. Version control is the proper solution.

4 What is Version Control?

Version control systems have existed since the 1970s, evolving through several generations. Early systems like SCCS and RCS tracked individual files. Centralized systems like CVS and Subversion tracked entire projects but required constant server connection. Git, introduced in 2005, pioneered the distributed model where every copy contains complete history.

Key capabilities that distinguish version control from simple backups:

- **Change tracking**: The system knows exactly which lines changed between any two versions, not just that "the file is different."
- **Attribution**: Every change records who made it and when, creating accountability and enabling questions about intent.
- **Reversibility**: Any previous state can be restored, either for the entire project or individual files.
- **Branching**: Parallel versions can exist simultaneously, enabling experimentation without risk.
- **Merging**: Changes from different sources can be combined intelligently, with conflict detection when the same lines were modified.

The comparison to Dropbox/Google Drive addresses a common misconception. Cloud storage provides synchronization and backup but lacks understanding of content. When conflicts occur, cloud storage creates duplicate files; version control identifies the specific conflicting lines and provides tools for resolution.

Technical note: The "track changes" analogy from word processors is useful but limited. Word processor tracking is linear and tied to a single document. Git tracks an entire project, supports non-linear history through branches, and enables multiple simultaneous editors with systematic merge capabilities.

5 Git: The Version Control Standard

Linus Torvalds created Git in 2005 after the Linux kernel project lost access to the proprietary BitKeeper system. Torvalds designed Git with specific goals:

- **Speed**: Most operations complete in milliseconds because they work locally.
- **Distributed architecture**: Every clone contains complete history, making the system resilient and enabling offline work.
- **Strong support for branching**: Creating and merging branches is fast and cheap, encouraging their use for feature development.
- **Data integrity**: Git uses cryptographic hashes (SHA-1) to identify content, making corruption detectable.

Git achieved dominance rapidly. By 2020, over 90% of developers used Git according to Stack Overflow surveys. For data science and biostatistics, Git has become the standard, supported by RStudio, VS Code, and most development environments.

The "distributed" concept deserves emphasis. In older centralized systems, the server held the authoritative history and clients held only current files. If the server failed, history was lost. With Git, every clone is complete—if GitHub disappeared tomorrow, every user would still have everything. This resilience matters for long-term research reproducibility.

Technical note: Git tracks content, not files. This means moving or renaming files does not lose history—Git recognizes that the content moved. It also means identical content is stored only once, making repositories efficient even with many similar files.

Reference: Chacon S, Straub B. *Pro Git* (2nd ed.). Apress. Available free at https://git-scm.com/book. The definitive Git reference.

6 Core Git Concepts

These five concepts form the vocabulary for all Git operations:

6.1 Repository (repo)

A repository is a project folder with Git tracking enabled. The tracking information lives in a hidden `.git` subdirectory. When you "clone" a repository, you copy both the project files and this tracking information. When you "initialize" a repository (`git init`), Git creates the `.git` directory in an existing folder.

6.2 Commit

A commit is a snapshot of your project at a specific moment. Each commit has:

- A unique identifier (40-character SHA-1 hash, usually abbreviated to 7 characters)
- A pointer to the previous commit (creating a chain of history)
- The state of all tracked files
- Metadata: author, timestamp, and commit message

Commits are immutable—once created, they never change. This immutability enables Git's reliability and makes history trustworthy.

6.3 Branch

A branch is a pointer to a commit, representing an independent line of development. The default branch is typically called "main" (historically "master"). Creating a branch creates a new pointer; the actual commits are shared until the branches diverge.

Branches enable parallel work. A feature branch lets you develop something new without affecting the main code. If the feature fails, delete the branch. If it succeeds, merge it.

6.4 Merge

Merging combines the changes from one branch into another. Git analyzes the commit history and automatically merges changes that do not conflict. When the same lines were modified differently, Git reports a conflict requiring manual resolution.

6.5 Remote

A remote is a reference to a repository on another computer, typically a server like GitHub. The conventional name "origin" refers to the repository you cloned from. Push sends local commits to a remote; pull retrieves commits from a remote.

Technical note: Understanding that branches are lightweight pointers, not copies, explains why branching is fast. Creating a branch writes 41 bytes (the hash plus a newline) to a file. This efficiency encourages liberal branch use.

7 Basic Git Workflow

The daily Git workflow follows a cycle: **Edit → Stage → Commit → Repeat**.

7.1 Checking Status

`git status` is the most frequently used command. It shows:

- Which branch you are on
- Whether you are ahead of or behind the remote
- Which files have been modified
- Which modifications are staged for commit
- Which files are untracked

Run `git status` frequently—before staging, before committing, after any operation you are unsure about. It provides orientation and prevents surprises.

7.2 Staging Changes

The staging area (also called the "index") is a holding area between your working files and the repository. `git add` moves changes into staging. This intermediate step enables crafting logical commits.

Why staging matters:

- You may have changed five files but want to commit related changes together.
- You may have experimental changes you are not ready to commit.
- You may want to commit part of a file's changes (using `git add -p`).

Staging provides control over what each commit contains, enabling meaningful history.

7.3 Committing

`git commit` creates a snapshot from the staged changes. The `-m` flag provides the commit message inline. Without `-m`, Git opens your configured text editor for a longer message.

Each commit should be a logical unit—one bug fix, one feature, one refactoring. Avoid commits that mix unrelated changes ("fix bug and update documentation and change formatting"). Atomic commits make history useful: you can revert one change without affecting others.

Technical note: `git add .` stages all changes in the current directory. This is convenient but dangerous—you may accidentally stage files you did not intend to commit. Prefer explicit file names or review with `git status` before committing.

8 Writing Good Commit Messages

Commit messages are documentation for your future self and collaborators. The Git community has converged on conventions:

8.1 Format

- **First line**: Summary under 50 characters, imperative mood ("Add feature" not "Added feature" or "Adding feature")
- **Blank line**: Separates summary from body
- **Body**: Longer explanation if needed, wrapped at 72 characters

8.2 Content

Good messages explain **why**, not just what. The code shows what changed; the message explains the reasoning.

Bad: "Changed threshold" **Good**: "Increase significance threshold to 0.01 per reviewer request"

For statistical analysis, messages should capture analytical decisions:

- Why a particular method was chosen
- What sensitivity analysis revealed
- What a collaborator or reviewer suggested
- What data issue was discovered and how it was handled

This creates an audit trail of analytical reasoning—essentially a lab notebook embedded in the repository history.

Technical note: The imperative mood ("Add feature") reads naturally when Git uses the message automatically: "This commit will: Add feature." It also matches Git's own generated messages (e.g., "Merge branch 'feature' ").

Reference: Beams C. "How to Write a Git Commit Message." https://cbea.ms/git-commit/. A widely-cited guide to commit message conventions.

9 Introduction to GitHub

GitHub, launched in 2008, transformed Git from a tool for large open-source projects into an accessible platform for all developers. Microsoft acquired GitHub in 2018 for $7.5 billion, reflecting its central role in software development.

9.1 What GitHub Adds

Git is the version control engine; GitHub provides:

- **Cloud hosting**: Repositories accessible from anywhere, backed up automatically.
- **Web interface**: Browse code, history, and changes through a browser.
- **Pull requests**: A mechanism for proposing, discussing, and reviewing changes before merging.
- **Issues**: Track bugs, feature requests, and tasks with discussion threads.
- **Actions**: Automate testing, building, and deployment when code changes.
- **Pages**: Host websites directly from repositories.
- **Organizations**: Manage team access and permissions.

9.2 Alternatives

GitHub is not the only option:

- **GitLab**: Similar features, can be self-hosted
- **Bitbucket**: Atlassian's offering, integrates with Jira
- **Gitea/Gogs**: Lightweight self-hosted alternatives

For this course, we use GitHub because of its prevalence and the GitHub Education program that provides free enhanced features for students.

Technical note: GitHub Education (https://education.github.com) provides free GitHub Pro accounts and access to the Student Developer Pack, which includes credits for various developer tools. Sign up with your UCSD email.

10 Connecting to GitHub

Several operations connect local Git to remote GitHub repositories:

10.1 Cloning

`git clone URL` downloads a repository and sets up the remote connection:

```
git clone https://github.com/username/repo.git
```

This creates a local directory with all files and complete history. The remote "origin" is configured automatically.

10.2 Adding a Remote

For an existing local repository, add a remote pointing to an empty GitHub repository:

```
git remote add origin https://github.com/username/repo.git
```

Then push your existing commits:

```
git push -u origin main
```

The `-u` flag sets up tracking so future `git push` and `git pull` commands default to this remote and branch.

10.3 Push and Pull

- `git push`: Send local commits to the remote
- `git pull`: Retrieve remote commits and merge them locally

The question about concurrent pushes addresses a common scenario. Git refuses to push if the remote has commits you lack—this prevents overwriting others' work. You must pull first, merge (resolving any conflicts), then push the result. This explicit handling ensures no work is silently lost.

Technical note: HTTPS URLs require username/password or token authentication. SSH URLs use key-based authentication, which is more convenient for frequent use. GitHub's documentation covers SSH setup: https://docs.github.com/en/authentication/connecting-to-github-with-ssh

11 Collaboration Workflow

Professional teams use branch-based workflows to coordinate work and maintain code quality. The basic pattern:

1. **Pull main**: Start with the latest shared code
2. **Create branch**: Work in isolation from main
3. **Commit frequently**: Small, logical commits
4. **Push branch**: Share your branch on GitHub
5. **Open pull request**: Propose merging your changes
6. **Review and discuss**: Teammates review the changes
7. **Merge**: After approval, integrate into main

11.1 Why Branches?

Direct commits to main cause problems:

- Incomplete work affects everyone immediately
- No opportunity for review before changes land
- Difficult to isolate and revert problematic changes

Branches provide isolation. Your work-in-progress stays on your branch until ready. Pull requests create a checkpoint for discussion and review.

11.2 Pull Requests

A pull request (PR) is not a Git feature—it is a GitHub collaboration feature. A PR:

- Shows all commits and changes in the branch
- Provides a discussion thread for feedback
- Enables line-by-line code review
- Can require approval before merging
- Creates documentation of the review process

For course projects, pull requests ensure teammates see and approve changes before they affect the shared analysis. This catches errors early and creates an audit trail.

Technical note: The name "pull request" reflects the original workflow: you request that the maintainer pull your changes. GitLab calls the same feature "merge request," which describes the outcome rather than the mechanism.

12 What to Track with Git

Not everything belongs in version control. The general principle: track source files that humans create; exclude generated files and sensitive data.

12.1 DO Track

- **Code**: .R, .Rmd, .qmd, .py, .do (Stata), .sas
- **Documentation**: README.md, analysis plans, codebooks
- **Configuration**: renv.lock, .Rprofile, Makefile, Dockerfile
- **Small data**: Reference tables, parameter files

12.2 DO NOT Track

- **Large data**: Datasets over a few MB (document location in README)
- **Sensitive data**: PHI, credentials, API keys (even if small)
- **Generated output**: HTML, PDF (usually—they can be regenerated)
- **Temporary files**: .Rhistory, .RData, .DS_Store, **pycache**
- **Environment-specific**: .Rproj.user, .vscode settings (usually)

12.3 The .gitignore File

The `.gitignore` file tells Git which files to ignore. Example for R projects:

```
# R temporary files
.Rhistory
.RData
.Rproj.user/

# Output files
*.html
*.pdf

# Data files
data/raw/
*.csv

# OS files
.DS_Store
Thumbs.db
```

Ignored files do not appear in `git status` and cannot be accidentally committed.

12.4 Handling Large Data

Large datasets should not go in Git (performance suffers and repositories become unwieldy), but collaborators need access. Options:

- **Document location**: Reference institutional servers or public repositories in README
- **Download scripts**: Include code to fetch public datasets
- **Git LFS**: Large File Storage extension for versioning large files (adds complexity)
- **DVC**: Data Version Control, a Git-like tool specifically for data pipelines

Technical note: Once a file is committed, it lives in Git history forever, even if deleted later. Never commit sensitive data—if you do accidentally, the repository history must be rewritten, which is disruptive for collaborators. Prevention is far easier than remediation.

13 Summary

Today's lecture covered the foundations of version control:

1. **The Problem**: Manual file versioning fails at scale and in collaboration. Version control solves this systematically.
2. **Git Fundamentals**: Repositories contain projects, commits capture snapshots, branches enable parallel work, merges combine changes.
3. **Workflow**: The edit-stage-commit cycle structures daily work. Good commit messages explain reasoning, not just changes.
4. **GitHub**: Cloud hosting plus collaboration features. Pull requests enable code review before merging.
5. **Collaboration**: Branch-based workflows isolate work-in-progress and require review before changes affect shared code.
6. **Best Practices**: Track code and configuration, not large data or secrets. Use .gitignore to keep repositories clean.

13.1 For Your Course Projects

- **Required**: Use Git for version control. All project work should be in GitHub repositories shared with your team and instructors.
- **Required**: Use branches and pull requests for team collaboration. Do not commit directly to main.
- **Expected**: Meaningful commit messages that document analytical decisions.

13.2 Coming Up

- **Tuesday, January 20**: Assign Project 1 (Penguins dataset). Teams will receive repository access and begin planning.

Technical note: If you have not already, ensure Git is installed and configured on your computer, and that you have a GitHub account. RStudio integrates with Git through Tools → Global Options → Git/SVN. Jenny Bryan's *Happy Git* has detailed setup instructions.

14 Additional References

14.1 Primary Documentation

- Git documentation: https://git-scm.com/doc
- GitHub documentation: https://docs.github.com
- GitHub Guides: https://guides.github.com

14.2 Books and Tutorials

- Bryan J. *Happy Git and GitHub for the useR*. https://happygitwithr.com/ Essential for R users, assumes no prior experience.
- Chacon S, Straub B. *Pro Git* (2nd ed.). https://git-scm.com/book Comprehensive reference, free online.
- Blischak JD, Davenport ER, Wilson G. (2016). A Quick Introduction to Version Control with Git and GitHub. *PLOS Computational Biology* 12(1):e1004668. https://doi.org/10.1371/journal.pcbi.1004668

14.3 Academic Papers

- Ram K. (2013). Git can facilitate greater reproducibility and increased transparency in science. *Source Code for Biology and Medicine* 8:7. https://doi.org/10.1186/1751-0473-8-7
- Perez-Riverol Y, et al. (2016). Ten Simple Rules for Taking Advantage of Git and GitHub. *PLOS Computational Biology* 12(7):e1004947. https://doi.org/10.1371/journal.pcbi.1004947

- Bryan J. (2018). Excuse Me, Do You Have a Moment to Talk About Version Control? *The American Statistician* 72(1):20-27. https://doi.org/10.1080/00031305.2017.1399928

14.4 Tools and Extensions

- GitHub Desktop: https://desktop.github.com/ Graphical Git client for Windows and macOS.
- GitKraken: https://www.gitkraken.com/ Cross-platform graphical Git client.
- Git LFS: https://git-lfs.github.com/ Extension for versioning large files.
- pre-commit: https://pre-commit.com/ Framework for managing Git hooks.